High-dimensional outcomes: sparse tabulations and the EM estimator

Why the balanced table stops working

`misclassifyr()` works from a **balanced** tabulation: one row for every combination of the regressor and the two recorded outcomes, including the combinations that never occur. That is `J^2 * K` rows.

For a six-category occupation scheme that is 216 rows and you never think about it. For anything at the granularity historical work actually cares about, the arithmetic turns hostile:

```
sizes <- data.frame(
  outcome = c("occupation, 6 groups", "US state", "occupation, 3-digit codes",
              "US county", "parish / township"),
  J = c(6, 50, 300, 3000, 15000)
)
sizes$K <- sizes$J
sizes$rows_balanced <- format(sizes$J^2 * sizes$K, big.mark = ",",
                              scientific = FALSE)
knitr::kable(sizes)
```

outcome	J	K	rows_balanced
occupation, 6 groups	6	6	216
US state	50	50	125,000
occupation, 3-digit codes	300	300	27,000,000
US county	3000	3000	27,000,000,000
parish / township	15000	15000	3,375,000,000,000

A county-level analysis would need a table with twenty-seven billion rows. That is not a table you build and then decide is too slow; it is a table that does not fit in memory on any machine you have.

But almost all of those rows are zero. In a linked sample of half a million records spread over 3,000 counties, at most 500,000 cells can be non-empty, out of twenty-seven billion. Two things follow, and this vignette is about both:

1. **Zero-count cells contribute nothing to the likelihood.** So you can drop them, if you keep track of where the surviving cells sit. That is `prep_misclassification_data(sparse = TRUE)`.
2. **The general estimator is still the wrong tool.** Even with a sparse table, `misclassifyr()` optimizes over a parameter vector containing all of Π — `J * K - 1` free parameters — using a quasi-Newton method with numerically differentiated gradients. At `J = 300` that is 89,999 parameters. For the record-linkage model specifically, there is a much better route: an expectation-maximization algorithm with closed-form updates, which is `misclassifyr_rl_em()`.

Sparse tabulations

`sparse = TRUE` keeps only the cells with positive counts and adds a `cell_idx` column recording each surviving cell's position in the balanced layout. `misclassifyr()` accepts either form and returns identical estimates — the sparse path exists purely to keep the tabulation from exploding.

```r
data(ancienregime)
occ <- c("Vagabond", "Metayer", "Journalier",
         "Petit Metiers", "Petite Bourgeoisie", "Haute Bourgeoisie")

args <- list(
  data      = ancienregime,
  outcome_1 = "son_occupation_1780",
  outcome_2 = "son_occupation_1770",
  regressor = "father_occupation_1750",
  X_names   = occ, Y1_names = occ, Y2_names = occ,
  weights   = "linked_weight"
)

dense  <- do.call(prep_misclassification_data, args)
sparse <- do.call(prep_misclassification_data, c(args, list(sparse = TRUE)))

c(dense_rows = nrow(dense$tab), sparse_rows = nrow(sparse$tab))
#>  dense_rows sparse_rows
#>         216         215
head(sparse$tab, 3)
#>            X       Y1       Y2         n cell_idx
#> 1   Vagabond Vagabond Vagabond  52.66729        1
#> 2    Metayer Vagabond Vagabond 675.84230        2
#> 3 Journalier Vagabond Vagabond 662.73423        3
```

Here the saving is trivial — one empty cell out of 216 — because this dataset has only six categories. The point is what `cell_idx` buys you: it lets the likelihood look up each observed cell's model probability without ever materializing the full grid.

```r
identical(dense$tab$n[sparse$tab$cell_idx], sparse$tab$n)
#> [1] TRUE
```

That equality is the entire contract. Everything downstream is unchanged.

The EM estimator for the record-linkage model

For the record-linkage model — each measure equals the truth with probability $1 - \alpha_m$ and is otherwise an independent draw from ρ_m — the likelihood of an observed cell factorizes into exactly four terms, one per configuration of the two links:

Configuration	Cell probability contribution
Both links correct	$(1-\alpha_1)(1-\alpha_2)\,\Pi_{y_1,x}\,\mathbf{1}\{y_1 = y_2\}$
First correct, second failed	$(1-\alpha_1)\,\alpha_2\,\rho_{2,y_2}\,\Pi_{y_1,x}$
First failed, second correct	$\alpha_1\,\rho_{1,y_1}\,(1-\alpha_2)\,\Pi_{y_2,x}$
Both failed	$\alpha_1\alpha_2\,\rho_{1,y_1}\rho_{2,y_2}\,\Pi_{+,x}$

Every M-step update is then a closed-form weighted average: α_m is the expected share of failed links, ρ_m the expected distribution of failed-link draws, Π the expected latent-cell counts. No numerical optimization, no Hessian, no J by J^2 matrix. Each iteration costs O(observed cells), and neither the misclassification matrices nor the balanced tabulation is ever formed.

The support of Π is fixed at initialization to the union of the observed (Y_1, X) and (Y_2, X) pairs. EM keeps zero cells at zero, so this is not a restriction so much as a recognition of one.

A `J = 300` example

Simulate a linked sample over 300 "counties": with probability 0.2 the son's true county differs from his father's, and each of the two recorded counties is a failed link with probability α_m.

```
rl_dgp <- function(J, N, alpha1, alpha2, stay = 0.8, seed = 23) {
  set.seed(seed)
  xi <- sample.int(J, N, replace = TRUE)                 # father's county
  moves <- sample.int(3, N, replace = TRUE)
  j <- ifelse(runif(N) < stay, xi, ((xi - 1 + moves) %% J) + 1)
  rho <- tabulate(j, J) / N
  y1 <- ifelse(runif(N) < alpha1, sample.int(J, N, replace = TRUE, prob = rho), j)
  y2 <- ifelse(runif(N) < alpha2, sample.int(J, N, replace = TRUE, prob = rho), j)
  tab <- aggregate(list(n = rep(1, N)),
                   by = list(X = xi, Y1 = y1, Y2 = y2), FUN = sum)
  list(tab = tab, rho = rho,
       Pi_true = unclass(table(factor(j, 1:J), factor(xi, 1:J))) / N)
}

d <- rl_dgp(J = 300, N = 4e5, alpha1 = 0.20, alpha2 = 0.35)

c(balanced_rows = 300^2 * 300, observed_cells = nrow(d$tab))
#>  balanced_rows observed_cells
#>       27000000         149957
round(nrow(d$tab) / (300^2 * 300), 4)                # fill rate
#> [1] 0.0056
```

Under one percent of the balanced grid is ever visited, and that is with a sample four hundred thousand records deep. Now estimate.

```
t0 <- Sys.time()
fit <- misclassifyr_rl_em(d$tab, J = 300, K = 300)
elapsed <- as.numeric(difftime(Sys.time(), t0, units = "secs"))

c(alpha1 = unname(fit$alpha["alpha1"]), alpha2 = unname(fit$alpha["alpha2"]),
  iterations = fit$n_iter, seconds = round(elapsed, 1))
#>     alpha1     alpha2 iterations    seconds
#>  0.1827974  0.3449091 57.0000000  3.1000000
```

Both rates recovered, in a couple of seconds, on a problem the general estimator cannot represent. The log likelihood increases monotonically, as EM guarantees — a useful sanity check on any run:

```
all(diff(fit$loglik_trace) > -1e-6)
#> [1] TRUE
```

The estimated joint distribution comes back on its support only, as a long data frame of `(j, i, p)` triples:

```
head(fit$Pi, 3)
#>   j i            p
#> 1 1 1 0.0025843635
#> 2 2 1 0.0002401933
#> 3 3 1 0.0001943520
nrow(fit$Pi)
#> [1] 82283

# Accuracy against the truth, on the support
p_true <- d$Pi_true[cbind(fit$Pi$j, fit$Pi$i)]
```

```
c(correlation = cor(fit$Pi$p, p_true),
  max_abs_err = max(abs(fit$Pi$p - p_true)))
#>  correlation  max_abs_err
#> 0.9999598066 0.0001038614
```

Also worth checking: $\hat{\rho}$, the rival distribution, against the true latent margin.

```
max(abs(fit$rho1 - d$rho))
#> [1] 0.0009750598
```

Two-step estimation with `alpha_fixed`

Estimating α and a 90,000-cell Π from the same data is uncomfortable. The number of parameters grows with the sample, which is the classic incidental-parameters setting: $\hat{\alpha}$ inherits bias from the noise in $\hat{\Pi}$, and its sampling distribution is not the textbook one.

The fix is to split the two jobs across two levels of aggregation.

1. **Coarse first step.** Aggregate the outcome to a level where cells are dense and the asymptotics are standard — states rather than counties, one-digit rather than three-digit occupation codes. Estimate α there. A false link is a false link regardless of how finely you code the outcome, so this $\hat{\alpha}$ transfers.
2. **Fine second step.** Estimate Π at the granularity you actually want, with α held fixed at the first-step value.

`alpha_fixed` implements step two:

```
# Step 1: aggregate 300 counties into 30 "states" and estimate alpha there
to_state <- function(v) ((v - 1) %/% 10) + 1
coarse <- aggregate(list(n = d$tab$n),
                    by = list(X  = to_state(d$tab$X),
                              Y1 = to_state(d$tab$Y1),
                              Y2 = to_state(d$tab$Y2)),
                    FUN = sum)

step1 <- misclassifyr_rl_em(coarse, J = 30, K = 30)

rbind(truth  = c(alpha1 = 0.20, alpha2 = 0.35),
      coarse = step1$alpha,
      fine   = fit$alpha) |> round(3)
#>        alpha1 alpha2
#> truth   0.200  0.350
#> coarse  0.198  0.348
#> fine    0.183  0.345
```

The coarse step lands on the truth, and slightly closer than the fine-grained free-α run — which is the incidental-parameters point in miniature. Now hold it fixed and re-estimate Π at full granularity:

```
step2 <- misclassifyr_rl_em(d$tab, J = 300, K = 300,
                            alpha_fixed = unname(step1$alpha))

step2$alpha                                    # held, not estimated
#>    alpha1    alpha2
#> 0.1979522 0.3481035
p2 <- d$Pi_true[cbind(step2$Pi$j, step2$Pi$i)]
c(free_alpha  = max(abs(fit$Pi$p   - p_true)),
  fixed_alpha = max(abs(step2$Pi$p - p2)))
```

```
#>   free_alpha  fixed_alpha
#> 1.038614e-04 9.452104e-05
```

One caveat on step one. Aggregation is only harmless if a failed link rarely lands inside the same coarse cell as the truth. That holds here, because the rival pool is spread over all 300 counties and only one in thirty draws falls in the right state. It fails if the linking algorithm itself matched on something correlated with the aggregation — if it matched on region, a rival is a same-region person, aggregation to regions hides most link failures, and $\hat{\alpha}$ from the coarse step is biased toward zero. In that case condition on the matching variable instead and share α across cells with `misclassifyr_rl_em_stacked()`; see `vignette("designing-misclassification-models")`.

You can also fix ρ_1 and ρ_2 the same way, through the `rho1` and `rho2` arguments, when you know the candidate pool the linking algorithm was drawing from — for instance because you can reconstruct it from the index the linker searched.

Separating true transitions from linkage error with `T2`

Here is a problem that only appears at high dimension. Suppose the outcome is county of residence and the two measures are ten years apart. The son's county in the second observation may differ from the first for two entirely different reasons: the link failed, or *he moved*. The plain record-linkage model cannot tell these apart, so it charges genuine migration to α_2 and reports a false-link rate that is too high.

The `T2` argument fixes this by giving the second measure its own true-transition operator:

$$\Delta^{(2)} = (1 - \alpha_2)\, T + \alpha_2\, \mathbf{1}\rho_2',$$

read as: a *correct* link observes a draw from T applied to the latent value — real migration between the two dates — while a failed link still draws from ρ_2. `T` is supplied as a sparse data frame with columns `j` (latent category), `l` (observed category) and `t` (probability), rows summing to one within `j`. It is a plug-in, held fixed and not estimated, which is the point: you discipline it with external information — a census migration question, published inter-county flows, a gravity model fitted to aggregate data — rather than asking the same data to identify both.

```
set.seed(31)
J <- 50; N <- 1.5e5; a1 <- 0.10; a2 <- 0.15

xi <- sample.int(J, N, replace = TRUE)
jj <- ifelse(runif(N) < 0.8, xi, ((xi + sample.int(4, N, replace = TRUE) - 1) %% J) + 1)
rho <- tabulate(jj, J) / N

# The son really moves between the two observation dates, 15% of the time
moved  <- sample.int(3, N, replace = TRUE)
l_true <- ifelse(runif(N) < 0.85, jj, ((jj - 1 + moved) %% J) + 1)

y1 <- ifelse(runif(N) < a1, sample.int(J, N, replace = TRUE, prob = rho), jj)
y2 <- ifelse(runif(N) < a2, sample.int(J, N, replace = TRUE, prob = rho), l_true)
tab <- aggregate(list(n = rep(1, N)), by = list(X = xi, Y1 = y1, Y2 = y2), FUN = sum)
```

The known migration structure, as a `T2` data frame:

```
T2 <- do.call(rbind, lapply(1:J, function(j) {
  data.frame(j = j,
             l = c(j, ((j - 1 + 1:3) %% J) + 1),
             t = c(0.85, rep(0.05, 3)))
}))
head(T2, 4)
```

```
#>   j l    t
#> 1 1 1 0.85
#> 2 1 2 0.05
#> 3 1 3 0.05
#> 4 1 4 0.05
```

Estimating with and without it:

```
with_T2    <- misclassifyr_rl_em(tab, J = J, K = J, T2 = T2)
without_T2 <- misclassifyr_rl_em(tab, J = J, K = J)

rbind(truth      = c(alpha1 = a1, alpha2 = a2),
      with_T2    = with_T2$alpha,
      without_T2 = without_T2$alpha) |> round(3)
#>            alpha1 alpha2
#> truth       0.100  0.150
#> with_T2     0.099  0.149
#> without_T2  0.103  0.264
```

Ignoring migration nearly doubles the estimated false-link rate for the second measure, while leaving the first — which is contemporaneous with the regressor and so unaffected — alone. If you then used that inflated $\hat{\alpha}_2$ to correct a coefficient, you would over-correct by the size of the migration rate.

Choosing between the estimators

	`misclassifyr()`	`misclassifyr_rl_em()`
Shape of Δ	any design you write	record-linkage slab only
Feasible `J`	tens	thousands
Tabulation	balanced, or sparse with `cell_idx`	observed cells only
Inference	delta method, posterior draws, Monte Carlo sets	point estimates; bootstrap or the two-step architecture
Controls	`controls` argument, estimated in parallel	`misclassifyr_rl_em_stacked()`, shared α
Extras	diagonal-dominance penalty, priors, trace plots	`alpha_fixed`, `rho1`/`rho2`, `T2`

The practical rule: if `J` is small enough that `J^2 * K` rows is a table you would be willing to print the dimensions of, use `misclassifyr()` and get the full inferential menu. If it is not, and your errors are linkage failures, use the EM estimator. If your errors at high dimension are *not* linkage failures, you are in genuinely hard territory and should think about aggregating the outcome before you think about estimators.

See `vignette("inference")` for what to report from either, and `vignette("designing-misclassification-models")` for how to choose the shape of Δ in the first place.